

Nota: Este material complementar, disponível em <https://prettore.github.io/lectures.html> representa uma cópia resumida de conteúdos bibliográficos disponíveis gratuitamente na Internet.

Algoritmos de Busca

Introdução.....	1
O Problema da Busca.....	2
Algoritmos de Busca em Vetores.....	2
1. Busca Linear (Sequencial).....	2
2. Busca Binária.....	3
3. Busca por Interpolação.....	4
4. Busca Exponencial.....	4
Algoritmos de Busca em Árvores.....	5
1. Árvore de Busca Binária (BST).....	5
2. Árvore AVL.....	6
3. Árvore Rubro-Negra.....	6
4. Árvore B e B+.....	7
Algoritmos de Busca em Grafos.....	7
1. Busca em Largura (BFS).....	7
2. Busca em Profundidade (DFS).....	8
3. Algoritmo de Dijkstra.....	9
4. Algoritmo A*.....	9
Algoritmos de Busca em Texto.....	9
1. Algoritmo de Força Bruta.....	9
2. Algoritmo KMP (Knuth-Morris-Pratt).....	9
3. Algoritmo de Boyer-Moore.....	10
4. Algoritmo de Rabin-Karp.....	10
Estruturas de Dados para Busca Eficiente.....	10
1. Tabelas Hash.....	10
2. Tries (Árvores de Prefixos).....	10
3. Árvores de Sufixos.....	11
Comparação entre Algoritmos de Busca.....	11
Aplicações dos Algoritmos de Busca.....	12
Boas Práticas na Implementação de Algoritmos de Busca.....	12
Conclusão.....	12
Referências.....	12

Introdução

Os algoritmos de busca são fundamentais na ciência da computação, pois permitem encontrar informações específicas em conjuntos de dados. Esses algoritmos são utilizados em diversas aplicações, desde a busca

de um elemento em um vetor até a navegação em grafos complexos para encontrar o caminho mais curto entre dois pontos. A eficiência desses algoritmos é crucial, especialmente quando lidamos com grandes volumes de dados.

Este material apresenta os principais algoritmos de busca, suas características, implementações, análises de complexidade e aplicações. Compreender esses algoritmos é essencial para qualquer profissional da área de computação, pois eles ilustram conceitos fundamentais de eficiência algorítmica e técnicas de programação.

O Problema da Busca

O problema da busca consiste em encontrar um elemento específico em uma coleção de dados. Formalmente, dado um conjunto de elementos e um valor alvo, o objetivo é determinar se o valor alvo está presente no conjunto e, em caso afirmativo, retornar sua posição ou referência.

Os algoritmos de busca podem ser classificados de várias formas, dependendo da estrutura de dados em que a busca é realizada, da estratégia utilizada e das garantias oferecidas. Alguns fatores importantes a considerar na escolha de um algoritmo de busca incluem:

1. Ordenação dos dados: Alguns algoritmos exigem que os dados estejam ordenados, enquanto outros funcionam com dados não ordenados.
2. Tamanho do conjunto de dados: Para conjuntos pequenos, algoritmos simples podem ser mais eficientes, enquanto para conjuntos grandes, algoritmos mais sofisticados são necessários.
3. Frequência das operações de busca: Se as buscas são frequentes, pode valer a pena investir em estruturas de dados mais complexas que acelerem as operações de busca.
4. Restrições de memória: Alguns algoritmos requerem espaço adicional, o que pode ser um fator limitante em sistemas com restrições de memória.

Algoritmos de Busca em Vetores

1. Busca Linear (Sequencial)

A busca linear é o algoritmo de busca mais simples. Ele examina cada elemento de um vetor, um após o outro, até encontrar o elemento desejado ou percorrer todo o vetor.

Implementação em C:

```
int linearSearch(int arr[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == x)  
            return i;  
    }  
    return -1; // Elemento não encontrado  
}
```

Complexidade:

- Tempo: $O(n)$ no pior caso e caso médio

- Espaço: $O(1)$

Características:

- Simples de implementar

- Funciona em vetores ordenados e não ordenados

- Ineficiente para grandes conjuntos de dados

- Adequado para pequenos conjuntos de dados ou buscas ocasionais

2. Busca Binária

A busca binária é um algoritmo eficiente para encontrar um elemento em um vetor ordenado. Ele funciona dividindo repetidamente o espaço de busca pela metade, comparando o elemento do meio com o valor alvo.

Implementação em C:

```
int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        int m = l + (r - l) / 2;

        // Verifica se x está presente no meio
        if (arr[m] == x)
            return m;

        // Se x é maior, ignora a metade esquerda
        if (arr[m] < x)
            l = m + 1;

        // Se x é menor, ignora a metade direita
        else
            r = m - 1;
    }

    // Elemento não encontrado
    return -1;
}
```

Complexidade:

- Tempo: $O(\log n)$ no pior caso e caso médio

- Espaço: $O(1)$ para implementação iterativa, $O(\log n)$ para implementação recursiva devido à pilha de chamadas

Características:

- Requer que o vetor esteja ordenado

- Muito mais eficiente que a busca linear para grandes conjuntos de dados

- Divide o espaço de busca pela metade a cada iteração

- Adequado para conjuntos de dados grandes e ordenados

3. Busca por Interpolação

A busca por interpolação é uma melhoria da busca binária para distribuições uniformes. Em vez de sempre dividir o espaço de busca pela metade, ela estima a posição do elemento com base em seu valor e na distribuição dos valores no vetor.

Implementação em C:

```
int interpolationSearch(int arr[], int n, int x) {
    int low = 0, high = n - 1;

    while (low <= high && x >= arr[low] && x <= arr[high]) {
        if (low == high) {
            if (arr[low] == x) return low;
            return -1;
        }

        // Fórmula de interpolação para estimar a posição
        int pos = low + (((double)(high - low) / (arr[high] - arr[low])) * (x - arr[low]));

        if (arr[pos] == x)
            return pos;

        if (arr[pos] < x)
            low = pos + 1;
        else
            high = pos - 1;
    }
    return -1;
}
```

Complexidade:

- Tempo: $O(\log \log n)$ no caso médio para distribuições uniformes, $O(n)$ no pior caso
- Espaço: $O(1)$

Características:

- Requer que o vetor esteja ordenado
- Mais eficiente que a busca binária para distribuições uniformes
- Menos eficiente para distribuições não uniformes
- Adequado para conjuntos de dados grandes, ordenados e uniformemente distribuídos

4. Busca Exponencial

A busca exponencial é útil quando o tamanho do vetor é ilimitado ou desconhecido. Ela funciona determinando um intervalo onde o elemento pode estar e, em seguida, aplicando a busca binária nesse intervalo.

Implementação em C:

```
int exponentialSearch(int arr[], int n, int x) {
    // Se o elemento estiver na primeira posição
    if (arr[0] == x)
        return 0;

    // Encontra o intervalo para a busca binária
    int i = 1;
    while (i < n && arr[i] <= x)
        i = i * 2;

    // Aplica busca binária no intervalo encontrado
    return binarySearch(arr, i/2, min(i, n-1), x);
}
```

Complexidade:

- Tempo: $O(\log n)$ no pior caso
- Espaço: $O(1)$ para implementação iterativa

Características:

- Requer que o vetor esteja ordenado
- Útil para vetores de tamanho desconhecido ou ilimitado
- Combina busca exponencial com busca binária
- Adequado para situações onde o elemento está próximo ao início do vetor

Algoritmos de Busca em Árvores

1. Árvore de Busca Binária (BST)

Uma árvore de busca binária é uma estrutura de dados em forma de árvore onde cada nó tem no máximo dois filhos (esquerdo e direito), e para cada nó, todos os elementos na subárvore esquerda são menores que o nó, e todos os elementos na subárvore direita são maiores.

Implementação em C:

```
struct Node {
    int key;
    struct Node *left, *right;
};

// Função para criar um novo nó
struct Node* newNode(int item) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->key = item;
    temp->left = temp->right = NULL;
}
```

```

    return temp;
}
// Função para buscar um valor na árvore
struct Node* search(struct Node* root, int key) {
    // Caso base: raiz é NULL ou a chave está presente na raiz
    if (root == NULL || root->key == key)
        return root;

    // A chave é maior que a chave da raiz
    if (root->key < key)
        return search(root->right, key);

    // A chave é menor que a chave da raiz
    return search(root->left, key);
}

```

Complexidade:

- Tempo: $O(h)$ onde h é a altura da árvore ($O(\log n)$ para árvores balanceadas, $O(n)$ no pior caso para árvores desbalanceadas)

- Espaço: $O(h)$ devido à pilha de chamadas recursivas

Características:

- Estrutura de dados dinâmica
- Eficiente para operações de busca, inserção e remoção
- Desempenho depende do balanceamento da árvore
- Adequado para conjuntos de dados dinâmicos que requerem buscas frequentes

2. Árvore AVL

A árvore AVL é uma árvore de busca binária balanceada, onde a diferença de altura entre as subárvores esquerda e direita de qualquer nó não pode ser maior que 1.

Características:

- Mantém-se balanceada automaticamente
- Garante operações de busca, inserção e remoção em $O(\log n)$
- Requer rotações para manter o balanceamento
- Adequado para conjuntos de dados dinâmicos com muitas operações de busca

3. Árvore Rubro-Negra

A árvore rubro-negra é outra forma de árvore de busca binária balanceada, que utiliza cores (vermelho e preto) para manter o balanceamento.

Características:

- Mantém-se balanceada automaticamente
- Garante operações de busca, inserção e remoção em $O(\log n)$

- Menos rotações que a árvore AVL, mas pode ser menos balanceada
- Adequado para conjuntos de dados dinâmicos com muitas operações de inserção e remoção

4. Árvore B e B+

As árvores B e B+ são estruturas de dados balanceadas projetadas para sistemas de armazenamento em disco, onde o acesso aos dados é mais caro que as operações em memória.

Características:

- Projetadas para minimizar operações de I/O
- Cada nó pode ter múltiplos filhos
- Mantêm todos os dados ordenados
- Adequadas para bancos de dados e sistemas de arquivos

Algoritmos de Busca em Grafos

1. Busca em Largura (BFS)

A busca em largura explora todos os vértices de um grafo em níveis, visitando primeiro todos os vértices adjacentes a um vértice antes de passar para o próximo nível.

Implementação em C (usando lista de adjacência):

```
void BFS(int graph[][MAX_VERTICES], int start, int vertices) {
    // Marca todos os vértices como não visitados
    bool visited[MAX_VERTICES] = {false};

    // Cria uma fila para BFS
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;

    // Marca o vértice atual como visitado e o enfileira
    visited[start] = true;
    queue[rear++] = start;

    while (front < rear) {
        // Desenfileira um vértice e o imprime
        int current = queue[front++];
        printf("%d ", current);

        // Obtém todos os vértices adjacentes do vértice desenfileirado
        // Se um adjacente não foi visitado, marca-o como visitado e o enfileira
        for (int i = 0; i < vertices; i++) {
            if (graph[current][i] && !visited[i]) {
                visited[i] = true;
                queue[rear++] = i;
            }
        }
    }
}
```

```

    }
  }
}

```

Complexidade:

- Tempo: $O(V + E)$ onde V é o número de vértices e E é o número de arestas
- Espaço: $O(V)$

Características:

- Encontra o caminho mais curto em grafos não ponderados
- Usa uma fila para processar os vértices
- Explora o grafo em níveis
- Adequado para encontrar componentes conectados e caminhos mais curtos

2. Busca em Profundidade (DFS)

A busca em profundidade explora o grafo seguindo um caminho até o fim antes de retroceder e explorar outros caminhos.

Implementação em C (usando lista de adjacência):

```

void DFSUtil(int graph[][MAX_VERTICES], int vertex, bool visited[], int vertices) {
    // Marca o vértice atual como visitado e o imprime
    visited[vertex] = true;
    printf("%d ", vertex);

    // Recorre para todos os vértices adjacentes a este vértice
    for (int i = 0; i < vertices; i++) {
        if (graph[vertex][i] && !visited[i])
            DFSUtil(graph, i, visited, vertices);
    }
}

void DFS(int graph[][MAX_VERTICES], int start, int vertices) {
    // Marca todos os vértices como não visitados
    bool visited[MAX_VERTICES] = {false};

    // Chama a função auxiliar recursiva para imprimir DFS
    DFSUtil(graph, start, visited, vertices);
}

```

Complexidade:

- Tempo: $O(V + E)$ onde V é o número de vértices e E é o número de arestas
- Espaço: $O(V)$ devido à pilha de chamadas recursivas

Características:

- Usa uma pilha (implícita na recursão) para processar os vértices

- Explora o grafo em profundidade
- Adequado para encontrar componentes conectados, ciclos e ordenação topológica

3. Algoritmo de Dijkstra

O algoritmo de Dijkstra encontra o caminho mais curto entre um vértice de origem e todos os outros vértices em um grafo ponderado com pesos não negativos.

Características:

- Encontra o caminho mais curto em grafos ponderados
- Usa uma fila de prioridade para selecionar o próximo vértice
- Não funciona com arestas de peso negativo
- Adequado para encontrar rotas mais curtas em mapas e redes

4. Algoritmo A*

O algoritmo A* é um algoritmo de busca informada que encontra o caminho mais curto entre um vértice de origem e um vértice de destino, usando uma heurística para guiar a busca.

Características:

- Combina as vantagens do algoritmo de Dijkstra e da busca gulosa
- Usa uma função heurística para estimar o custo restante
- Mais eficiente que o algoritmo de Dijkstra para encontrar um caminho específico
- Adequado para jogos, navegação de robôs e planejamento de rotas

Algoritmos de Busca em Texto

1. Algoritmo de Força Bruta

O algoritmo de força bruta para busca em texto compara o padrão com todas as possíveis substrings do texto.

Complexidade:

- Tempo: $O(m * n)$ onde m é o comprimento do padrão e n é o comprimento do texto
- Espaço: $O(1)$

Características:

- Simples de implementar
- Ineficiente para textos longos
- Não requer pré-processamento
- Adequado para padrões curtos ou buscas ocasionais

2. Algoritmo KMP (Knuth-Morris-Pratt)

O algoritmo KMP evita comparações redundantes usando um array de prefixo que armazena informações sobre o próprio padrão.

Características:

- Evita retroceder no texto

- Requer pré-processamento do padrão
- Tempo de execução $O(m + n)$
- Adequado para busca de padrões em textos longos

3. Algoritmo de Boyer-Moore

O algoritmo de Boyer-Moore usa duas heurísticas (regra do caractere ruim e regra do sufixo bom) para pular comparações desnecessárias.

Características:

- Pode pular várias posições de uma vez
- Requer pré-processamento do padrão
- Geralmente mais rápido que KMP na prática
- Adequado para busca de padrões longos em textos longos

4. Algoritmo de Rabin-Karp

O algoritmo de Rabin-Karp usa hashing para comparar o padrão com substrings do texto.

Características:

- Usa funções hash para comparações rápidas
- Bom para busca de múltiplos padrões
- Tempo médio de execução $O(m + n)$
- Adequado para detecção de plágio e busca de múltiplos padrões

Estruturas de Dados para Busca Eficiente

1. Tabelas Hash

As tabelas hash são estruturas de dados que mapeiam chaves para valores usando uma função hash, permitindo acesso direto aos elementos.

Características:

- Tempo médio de busca, inserção e remoção $O(1)$
- Requer uma boa função hash para evitar colisões
- Não mantém os elementos ordenados
- Adequado para dicionários, caches e conjuntos

2. Tries (Árvores de Prefixos)

As tries são estruturas de dados especializadas para armazenar um conjunto de strings, onde cada nó representa um prefixo comum.

Características:

- Tempo de busca, inserção e remoção $O(m)$ onde m é o comprimento da string
- Eficiente para operações de prefixo

- Usa mais memória que outras estruturas
- Adequado para autocompletar, verificação ortográfica e roteamento IP

3. Árvores de Sufixos

As árvores de sufixos são estruturas de dados que armazenam todos os sufixos de uma string, permitindo buscas eficientes de substrings.

Características:

- Permite busca de substrings em $O(m)$ onde m é o comprimento da substring
- Requer pré-processamento do texto
- Usa mais memória que outras estruturas
- Adequado para análise de DNA, compressão de dados e busca de padrões

Comparação entre Algoritmos de Busca

A escolha do algoritmo de busca adequado depende de vários fatores, como o tipo de dados, a frequência das operações de busca, as restrições de memória e a necessidade de manter os dados ordenados. A tabela a seguir resume as características dos principais algoritmos:

Algoritmo	Estrutura de Dados	Melhor Caso	Caso Médio	Pior Caso	Espaço	Ordenado?
Busca Linear	Vetor	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Não
Busca Binária	Vetor	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Sim
Busca por Interpolação	Vetor	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$	Sim
Árvore de Busca Binária	Árvore	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$	-
Árvore AVL	Árvore	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$	-
Tabela Hash	Hash	$O(1)$	$O(1)$	$O(n)$	$O(n)$	Não
BFS (Busca em Largura)	Grafo	$O(1)$	$O(V + E)$	$O(V + E)$	$O(V)$	-
DFS (Busca em Profundidade)	Grafo	$O(1)$	$O(V + E)$	$O(V + E)$	$O(V)$	-

Aplicações dos Algoritmos de Busca

Os algoritmos de busca têm inúmeras aplicações em diversas áreas da computação:

1. Bancos de dados: Para indexação e consultas eficientes.
2. Sistemas operacionais: Para gerenciamento de processos e recursos.
3. Redes de computadores: Para roteamento e gerenciamento de pacotes.
4. Inteligência artificial: Para busca de soluções em espaços de estados.
5. Processamento de texto: Para busca de padrões e verificação ortográfica.
6. Computação gráfica: Para detecção de colisões e renderização.
7. Bioinformática: Para análise de sequências de DNA e proteínas.
8. Sistemas de recomendação: Para encontrar itens similares.

Boas Práticas na Implementação de Algoritmos de Busca

1. Escolha o algoritmo adequado para o problema específico, considerando o tipo de dados, a frequência das operações de busca e as restrições de memória.
2. Para pequenos conjuntos de dados, algoritmos simples como a busca linear podem ser mais eficientes devido à baixa sobrecarga.
3. Para grandes conjuntos de dados ordenados, prefira algoritmos como a busca binária ou estruturas de dados como árvores balanceadas.
4. Para buscas frequentes, considere estruturas de dados especializadas como tabelas hash ou tries.
5. Implemente otimizações específicas para o seu caso de uso, como cache de resultados ou pré-processamento de dados.
6. Teste o desempenho do algoritmo com diferentes tamanhos e distribuições de dados.

Conclusão

Os algoritmos de busca são ferramentas fundamentais na ciência da computação, com aplicações em praticamente todas as áreas que envolvem processamento de dados. Compreender as características, vantagens e limitações de cada algoritmo permite escolher a solução mais adequada para cada problema específico.

Embora existam algoritmos com complexidade teórica ótima para diferentes cenários, não existe um "melhor" algoritmo universal. A escolha depende das características dos dados e dos requisitos específicos da aplicação.

O estudo dos algoritmos de busca também fornece insights valiosos sobre técnicas de projeto de algoritmos, como divisão e conquista, uso de estruturas de dados auxiliares e análise de complexidade, que são aplicáveis a uma ampla gama de problemas computacionais.

Referências

-  Livros e Apostilas

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. MIT Press.
- SEDGEWICK, R.; WAYNE, K. Algorithms. Addison-Wesley Professional.
- KNUTH, D. E. The Art of Computer Programming, Volume 3: Sorting and Searching. Addison-Wesley.
- ZIVIANI, N. Projeto de Algoritmos com Implementações em Pascal e C. Cengage Learning.
-  Recursos Online
 - Khan Academy - Implementação de busca binária de um array - <https://pt.khanacademy.org/computing/computer-science/algorithms/binary-search/a/implementing-binary-search-of-an-array>
 - Medium - Dart — Algoritmos de Busca: Binária e Linear - <https://medium.com/@hlfddev/dart-algoritmo-de-busca-bin%C3%A1ria-e-linear-19cb7df6a147>
 - Wikipedia - Categoria:Algoritmos de busca - https://pt.wikipedia.org/wiki/Categoria:Algoritmos_de_busca
 - IC-Unicamp - Algoritmos de Busca - <https://ic.unicamp.br/~mc102/aulas/aula11.pdf>
-  Vídeos e Cursos
 - Coursera - Algorithms, Part I (Princeton University)
 -  Algoritmos de Busca - Linear e Binária

Isenção de Responsabilidade:

Os autores deste documento não reivindicam a autoria do conteúdo original compilado das fontes mencionadas. Este documento foi elaborado para fins educativos e de referência, e todos os créditos foram devidamente atribuídos aos respectivos autores e fontes originais.

Qualquer utilização comercial ou distribuição do conteúdo aqui compilado deve ser feita com a devida autorização dos detentores dos direitos autorais originais. Os compiladores deste documento não assumem qualquer responsabilidade por eventuais violações de direitos autorais ou por quaisquer danos decorrentes do uso indevido das informações contidas neste documento.

Ao utilizar este documento, o usuário concorda em respeitar os direitos autorais dos autores originais e isenta os compiladores de qualquer responsabilidade relacionada ao conteúdo aqui apresentado.